# HW6

CSCE 5218 - Deep Learning

Siva Kumar Reddy Surasani || 11702175

## Question 1: AlexNet Architecture and Parameter Calculation

### (a) Output dimensions of CONV1

For a convolution with input size $N$, kernel size $K$, padding $P$, and stride $S$, each spatial output dimension is

$$N_{\text{out}} = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1.$$

Substituting N, K, P, and S

$$N_{\text{out}} = \left\lfloor \frac{227 - 11}{4} \right\rfloor + 1 = 54 + 1 = 55.$$

Each of the 96 filters produces one output feature map. Therefore, using height $\times$ width $\times$ channels,

$$\boxed{\text{CONV1 output} = 55 \times 55 \times 96.}$$

### (b) Number of trainable parameters in CONV1

Each filter spans all three input channels, so the number of weights per filter is

$$11 \times 11 \times 3 = 363.$$

Including one bias per filter,

$$\text{Parameters per filter} = 363 + 1 = 364.$$

For 96 filters,

$$\boxed{\text{Total parameters} = 364 \times 96 = 34{,}944.}$$

These consist of 34848 weights and 96 biases. The weights are shared across spatial positions, so the parameter count is not multiplied by the number of output positions.

### (c) Output dimensions of MAX POOL1

Assuming zero padding, the input spatial size is 55, the pooling window is $3 \times 3$, and the stride is 2. Therefore,

$$N_{\text{out}} = \left\lfloor \frac{55 - 3}{2} \right\rfloor + 1 = 26 + 1 = 27.$$

Pooling operates independently on each feature map and preserves the number of channels:

$$\boxed{\text{MAX POOL1 output} = 27 \times 27 \times 96.}$$

**(d) Why max pooling has no trainable parameters**

Max pooling selects the largest value within each local window. This is a fixed operation that does not require learned weights or biases. Its window size and stride are hyperparameters, not trainable parameters.

In this example, pooling reduces the spatial dimensions from $55 \times 55$ to $27 \times 27$, while the feature depth remains 96. It reduces computation in subsequent layers and provides some tolerance to small changes in the position of strong activations.

## Question 2: VGGNet and Small Convolutional Filters

**(a)** Each $3 \times 3$ layer increases the receptive-field size by two pixels in each spatial dimension.

Starting with a single input position,

$$r_0 = 1, \qquad r_\ell = r_{\ell-1} + (3-1).$$

Consequently,

$$r_1 = 3, \qquad r_2 = 5, \qquad r_3 = 7.$$

Thus, an output unit after three layers can depend on a $7 \times 7$ region of the original input:

$$\boxed{\text{Effective receptive field} = 7 \times 7.}$$

**(b) Comparison of convolutional weights**

Each $3 \times 3$ convolution with $C$ input channels and $C$ output channels has

$$3 \times 3 \times C \times C = 9C^2$$

weights.

For three such layers,

$$\boxed{W_{\text{small}} = 3(9C^2) = 27C^2.}$$

A single $7 \times 7$ layer has

$$\boxed{W_{\text{large}} = 7 \times 7 \times C \times C = 49C^2.}$$

The stacked design saves

$$49C^2 - 27C^2 = 22C^2$$

weights, corresponding to

$$\frac{22}{49} \times 100 \approx 44.9\%$$

fewer convolutional weights.

**(c) Two advantages of stacking small filters**

**First, parameter efficiency:** Three $3 \times 3$ layers achieve a $7 \times 7$ receptive field using $27C^2$ weights, compared with $49C^2$ for one $7 \times 7$ layer.

**Second, additional nonlinear processing:** When activations such as ReLU are inserted between the convolutional layers, the stack performs multiple nonlinear transformations. This enables more expressive feature extraction than one convolution followed by one activation.

**(d) Why depth can improve representational capacity**

Receptive field size describes which input pixels can influence an output, but it does not describe the complexity of the function computed from those pixels.

A deeper stack with intermediate nonlinear activations can progressively combine simple features into more complex representations. For example, one layer may detect edges, another may combine edges into textures or shapes, and a later layer may combine these into task-relevant patterns.

Therefore, two architectures can have the same receptive field size while differing in the functions they can represent. This advantage relies on intermediate nonlinearities; a stack of purely linear convolutions remains a linear transformation.

## Question 3: ResNet and Residual Learning

**(a) The degradation problem**

The degradation problem occurs when adding layers to a plain CNN causes its training error to increase, despite the deeper model having greater representational capacity.

In principle, the additional layers could preserve the shallower network's behavior by learning identity mappings. In practice, optimization may fail to find such a solution.

This is not simply overfitting. Overfitting typically produces low training error but worse performance on unseen data. Degradation includes worse training performance, indicating an optimization difficulty.

**(b) Residual mapping and block output**

If the desired mapping is $H(x)$, the residual mapping is

$$\boxed{F(x) = H(x) - x.}$$

The output of a residual block with an identity shortcut is

$$\boxed{y = F(x) + x.}$$

**(c) Why learning the residual may be easier**

If an identity mapping is desirable, then

$$H(x) = x.$$

The required residual becomes

$$F(x) = H(x) - x = 0.$$

Thus, the residual branch only needs to learn a zero or near-zero correction. The shortcut already preserves the input.

More generally, residual learning allows the convolutional layers to learn changes to the existing representation rather than reconstruct the entire desired mapping. This can make optimization easier, although it does not guarantee that every residual model will outperform every plain network.

**(d) How identity shortcuts help gradients**

For the simplified scalar residual block $y = x + F(x)$, the gradient is:

$$\frac{dL}{dx} = \frac{dL}{dy}\left(1 + F'(x)\right).$$

The term 1 comes from the identity shortcut and provides a direct contribution to the gradient. This helps gradients propagate even when the residual branch's derivative is small, making very deep networks easier to train.

## Question 4: Improvements to Deep CNN Architectures

**(a) Wide Residual Networks**

Wide Residual Networks increase the number of channels in residual blocks, often using fewer layers than very deep, narrow ResNets. Increasing width allows a layer to learn a larger collection of features.

Increasing depth adds sequential operations: a later layer must wait for earlier layers. Increasing width creates more computation within a layer that can be parallelized on suitable hardware. Therefore, a wider, shallower model can sometimes make better use of a GPU and train faster than a much deeper, narrow model.

**(b) Stochastic depth during training and inference**

During training, stochastic depth randomly skips some residual branches while keeping their identity shortcuts. If a branch is kept, the block produces $y = x + F(x)$. If it is skipped, the block simply produces $y = x$.

During inference, all residual branches are used. Under the convention of using unscaled branches during training, each branch's contribution is multiplied by its survival probability $p$:

$$y = x + pF(x).$$

For example, if $x = 5$, $F(x) = 2$, and $p = 0.5$, the training output is either 7 or 5, each with equal probability. During inference, the output is $5 + 0.5(2) = 6$.

Stochastic depth makes the network shorter on average during training, reducing computation and providing shorter paths for gradient propagation. It also acts as regularization by training different sub networks with shared parameters, which can reduce overfitting.

**(c) DenseNet connectivity**

Within a DenseNet dense block, each layer receives the concatenated feature maps of every preceding layer:

$$\boxed{x_\ell = concat([x_0, x_1, \ldots, x_{\ell-1}]).}$$

Thee above differs from ResNet, which uses element wise addition.

Earlier features remain directly accessible to later layers, encouraging feature reuse rather than repeatedly relearning similar information. The connections also create short routes between earlier layers and the loss, helping alleviate vanishing gradients.

If the initial input has $C_0$ channels and each layer adds $k$ new channels, the accumulated representation after $L$ layers has

$$C_0 + Lk$$

channels, before any transition-layer compression. Here $k$ is called the growth rate.

## Question 5: Understanding and Visualizing CNNs

### (a) Early-layer versus deeper-layer features

Early convolutional layers commonly learn simple local patterns, including oriented edges, color contrasts, and basic textures. Their receptive fields are relatively small.

Deeper layers combine earlier features over larger receptive fields. They may respond to complex textures, shapes, object parts, or task-specific combinations of patterns.

These are general tendencies rather than a strict rule. A feature channel does not necessarily correspond to one clearly interpretable object or concept.

### (b) Occlusion experiments

An occlusion experiment measures how covering different image regions changes a model's prediction.

First, the original target-class score is recorded. A small patch is then covered or replaced, and the score is recomputed. This procedure is repeated across image positions.

For an occluded image $x_{\mathrm{occ}(r)}$ at region $r$, a simple importance measure is

$$\Delta s_r = s_c(x) - s_c(x_{\mathrm{occ}(r)}).$$

A large positive $\Delta s_r$ suggests that region $r$ supports the target-class prediction. A negative value indicates that removing the region increased the score.

The score changes can be displayed as an importance map. However, the occluder can introduce unnatural patterns, and interactions between regions mean that this is a sensitivity analysis rather than a complete explanation of the model.

### (c) Activation maximization

Activation maximization helps us understand what patterns a neuron or feature channel responds to. We start with random noise or an existing image and gradually change its pixels to increase a selected activation.

The model's weights remain fixed. Only the input image changes, using gradient ascent:

$$x_{\mathrm{new}} = x + \eta \nabla_x a(x),$$

where $a(x)$ is the selected activation and $\eta$ controls the size of each update.

The gradient indicates how to change the pixels to increase the activation. After repeated updates, the image reveals patterns that strongly activate the selected unit. Constraints can help reduce noise and keep pixel values within a valid range.

**(d) High confidence on unrecognizable images**

A CNN can assign high confidence to an image that humans do not recognize because the learned decision function may respond strongly to patterns that differ from human semantic understanding.

For example, softmax produces relative class scores:

$$p_c = \frac{e^{z_c}}{\sum_j e^{z_j}}.$$

If one logit is much larger than the others, its softmax probability can be close to one, even when the input is unlike the training data.

Therefore, confidence alone does not establish correctness, familiarity of the input, or reliable understanding. Predictions should be assessed using labeled evaluation data, calibration, and robustness or out-of-distribution testing when appropriate.